repository) contains non-free elements, especially hardware drivers that depend on non-free firmware. It will ask you to install additional non-free software that it doesn't contain.

In fact, there is a project (unfortunately, not very active!) involved in the process of removing software that is included without source code; say, with obfuscated or obscured source code. It is the Linux-libre project of FSF-LA.

Now let's have a look at the automated process (for an *exploded* tree) that does it!

```
kver=2.6.21 extra=0++

case $1 in
--force) die () { echo ERROR: "$@": ignored >&2; }; shift;;
*) die () { echo "$@" >&2; exit 1; };;
esac

if unifdef -Utest /dev/null; then :; else
   die unifdef is required
fi

check=`echo $0 | sed 's,/[^/]*$,,'`/deblob-check
if [ ! -f $check ] ; then
  echo optional deblob-check missing, will remove entire files >&2
  have_check=false
else
  have_check=:
fi
```

Those who are good with shell programming can follow the steps easily (please refer to Segment 1 of the 'Voyage' series for shell programming related queries).

Now you can see how it performs the locating process:

```
clean_file () {
     #$1 = filename
     if test ! -f $1; then
          die $1 does not exist, something is wrong
     fi
     rm -v $1
}

check_changed () {
     if test ! -f $1; then
          die $1 does not exist, something is wrong
     elif cmp $1.deblob $1 > /dev/null; then
          die $1 did not change, something is wrong
     fi
     mv $1.deblob $1
}

clean_blob () {
     #$1 = filename
     if $have_check; then
       if test ! -f $1; then
          die $1 does not exist, something is wrong
       fi
       name=$1
       echo Removing blobs from $name
       set fnord "$@" -d
       shift 2
       $check "$@" -i linux-$kver $name > $name.deblob
       check_changed $name
     else
       clean_file $1
     fi
}

clean_kconfig () {
```

Inode interface

- *create()*: creates file in the directory
- *lookup()*: finds files by name, in a directory
- *link()/symlink()/unlink()/readlink()/follow_link()*: manages filesystem links
- *mkdir()/rmdir()*: creates/removes sub-directories
- *mknod()*: creates a directory or file
- *readpage()/writepage()*: reads or writes a page of physical memory to backing store
- *truncate()*: sets the length of a file to zero
- *permission()*: checks to see if a user process has permission to execute a given operation
- *smap()*: maps a logical file block to a physical device sector
- *bmap()*: maps a logical file block to a physical device block
- *rename()*: renames a file/directory

File interface

- *open()/release()*: to open/close the file
- *read()/write()*: read the file/write to the file
- *select()*: waits until the file is in a given state
- *lseek()*: moves to a particular offset in the file (if supported)
- *mmap()*: maps a region of the file into the virtual memory of the user process
- *fsync()/fasync()*: synchronises memory buffers with physical device
- *readdir*: reads the files that are pointed by the directory file
- *ioctl*: sets file attributes
- *check_media_change*: checks if a removable media has been removed
- *revalidate*: verifies that all the cached information is valid